

Task Lapindromes

Aim :

To write a Java program to check whether a given string is a lapindrome

Algorithm :

Step 1 Start the program

Step 2 Read the number of test cases

Step 3 Read one string for each test case

Step 4 Find the length of the string

Step 5 Split the string into two halves

Step 6 Ignore the middle character if the length is odd

Step 7 Count the frequency of characters in the left half

Step 8 Subtract the frequency of characters in the right half

Step 9 If all frequencies become zero display YES

Step 10 Otherwise display NO

Step 11 Stop the program

Program :

```
import java.util.Scanner;
```

```
public class Main {  
    static boolean isLapindrome(String s) {  
        int[] count = new int[26];  
        int n = s.length();  
        for (int i = 0; i < n / 2; i++) {  
            count[s.charAt(i) - 'a']++;  
        }  
    }  
}
```

```

    }

    for (int i = (n + 1) / 2; i < n; i++) {
        count[s.charAt(i) - 'a']--;
    }

    for (int value : count) {
        if (value != 0) {
            return false;
        }
    }

    return true;
}

```

```

public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);

    int t = sc.nextInt();
    while (t-- > 0) {
        String s = sc.next();

        if (isLapindrome(s)) {
            System.out.println("YES");
        } else {
            System.out.println("NO");
        }
    }

    sc.close();
}
}

```

Sample Input :

6

gaga

abcde

rotor

xyzxy

abbaab

ababc

Sample Output :

YES

NO

YES

YES

NO

NO

Determine If String Halves Are Alike

Aim :

To write a Java program to check whether two halves of a string have the same number of vowels

Algorithm :

Step 1 Start the program

Step 2 Read the string s

Step 3 Find the middle position of the string

Step 4 Count vowels in the first half of the string

Step 5 Count vowels in the second half of the string

Step 6 Compare both vowel counts

Step 7 Display true if both counts are equal

Step 8 Otherwise display false

Step 9 Stop the program

Program :

```
import java.util.Scanner;
```

```
public class Main {
```

```
    static boolean isVowel(char ch) {
```

```
        return ch == 'a' || ch == 'e' || ch == 'i' || ch == 'o' || ch == 'u'
```

```
            || ch == 'A' || ch == 'E' || ch == 'I' || ch == 'O' || ch == 'U';
```

```
    }
```

```
    static boolean halvesAreAlike(String s) {
```

```
        int mid = s.length() / 2;
```

```
int firstCount = 0;
int secondCount = 0;
for (int i = 0; i < mid; i++) {
    if (isVowel(s.charAt(i))) {
        firstCount++;
    }
}
for (int i = mid; i < s.length(); i++) {
    if (isVowel(s.charAt(i))) {
        secondCount++;
    }
}
return firstCount == secondCount;
}
```

```
public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    String s = sc.next();
    System.out.println(halvesAreAlike(s));
    sc.close();
}
}
```

Sample Input :

book

Sample Output :

true

Sample Input :

textbook

Sample Output :

false

Compare The Triplets

Aim :

To write a Java program to compare the ratings of Alice and Bob and calculate their scores

Algorithm :

Step 1 Start the program

Step 2 Read three ratings of Alice

Step 3 Read three ratings of Bob

Step 4 Initialize Alice score as zero

Step 5 Initialize Bob score as zero

Step 6 Compare each rating of Alice with the corresponding rating of Bob

Step 7 If Alice rating is greater increase Alice score by one

Step 8 If Bob rating is greater increase Bob score by one

Step 9 If both ratings are equal do not change any score

Step 10 Display Alice score and Bob score

Step 11 Stop the program

Program :

```
import java.util.Scanner;
```

```
public class Main {  
    static int[] compareTriplets(int[] a, int[] b) {  
        int alice = 0;  
        int bob = 0;  
        for (int i = 0; i < 3; i++) {  
            if (a[i] > b[i]) {
```

```
        alice++;
    } else if (a[i] < b[i]) {
        bob++;
    }
}
return new int[]{alice, bob};
}
```

```
public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    int[] a = new int[3];
    int[] b = new int[3];
    for (int i = 0; i < 3; i++) {
        a[i] = sc.nextInt();
    }
    for (int i = 0; i < 3; i++) {
        b[i] = sc.nextInt();
    }
    int[] result = compareTriplets(a, b);
    System.out.println(result[0] + " " + result[1]);
    sc.close();
}
}
```


Sample Input :

5 6 7

3 6 10

Sample Output :

1 1

Contains Duplicate

Aim :

To write a Java program to check whether any value appears at least twice in an integer array

Algorithm :

Step 1 Start the program

Step 2 Read the size of the array

Step 3 Read all array elements

Step 4 Create a HashSet to store visited elements

Step 5 Visit each element of the array

Step 6 If the element is already present in the set return true

Step 7 Otherwise add the element to the set

Step 8 If no repeated element is found return false

Step 9 Display the result

Step 10 Stop the program

Program :

```
import java.util.HashSet;
```

```
import java.util.Scanner;
```

```
import java.util.Set;
```

```
public class Main {
```

```
    static boolean containsDuplicate(int[] nums) {
```

```
        Set<Integer> seen = new HashSet<>();
```

```
        for (int num : nums) {
```

```
            if (seen.contains(num)) {
```

```
        return true;
    }
    seen.add(num);
}
return false;
}

public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    int n = sc.nextInt();
    int[] nums = new int[n];
    for (int i = 0; i < n; i++) {
        nums[i] = sc.nextInt();
    }
    System.out.println(containsDuplicate(nums));
    sc.close();
}
}
```

Sample Input :

4

1 2 3 1

Sample Output :

true

Sample Input :

4

1 2 3 4

Sample Output :

false

Sample Input :

10

1 1 1 3 3 4 3 2 4 2

Sample Output :

true

Time Conversion

Aim :

To write a Java program to convert time from twelve hour format to twenty four hour format

Algorithm :

Step 1 Start the program

Step 2 Read the time string

Step 3 Extract the period from the last two characters

Step 4 Extract the hour from the first two characters

Step 5 If the period is AM and the hour is twelve change the hour to zero

Step 6 If the period is PM and the hour is not twelve add twelve to the hour

Step 7 Join the converted hour with the remaining time part

Step 8 Display the time in twenty four hour format

Step 9 Stop the program

Program :

```
import java.util.Scanner;
```

```
public class Main {  
    static String timeConversion(String s) {  
        String period = s.substring(8);  
        int hour = Integer.parseInt(s.substring(0, 2));  
        if (period.equals("AM")) {  
            if (hour == 12) {  
                hour = 0;  
            }  
        }  
    }  
}
```

```
    } else {  
        if (hour != 12) {  
            hour += 12;  
        }  
    }  
    return String.format("%02d", hour) + s.substring(2, 8);  
}  
  
public static void main(String[] args) {  
    Scanner sc = new Scanner(System.in);  
    String s = sc.next();  
    System.out.println(timeConversion(s));  
    sc.close();  
}  
}
```

Sample Input :

07:05:45PM

Sample Output :

19:05:45

Move Zeroes

Aim :

To write a Java program to move all zeroes to the end of an array while maintaining the order of non zero elements

Algorithm :

Step 1 Start the program

Step 2 Read the size of the array

Step 3 Read all array elements

Step 4 Set an index position as zero

Step 5 Visit each element of the array

Step 6 If the element is not zero place it at the current index position

Step 7 Increase the index position by one

Step 8 Fill the remaining positions with zero

Step 9 Display the modified array

Step 10 Stop the program

Program :

```
import java.util.Scanner;
```

```
public class Main {  
    static void moveZeroes(int[] nums) {  
        int index = 0;  
        for (int num : nums) {  
            if (num != 0) {  
                nums[index] = num;  
                index++;  
            }  
        }  
    }  
}
```

```
    }  
}  
while (index < nums.length) {  
    nums[index] = 0;  
    index++;  
}  
}
```

```
public static void main(String[] args) {  
    Scanner sc = new Scanner(System.in);  
    int n = sc.nextInt();  
    int[] nums = new int[n];  
    for (int i = 0; i < n; i++) {  
        nums[i] = sc.nextInt();  
    }  
    moveZeroes(nums);  
    for (int i = 0; i < n; i++) {  
        System.out.print(nums[i]);  
        if (i < n - 1) {  
            System.out.print(" ");  
        }  
    }  
    sc.close();  
}
```


Sample Input :

5

0 1 0 3 12

Sample Output :

1 3 12 0 0

Sample Input :

1

0

Sample Output :

0

Diagonal Difference

Aim :

To write a Java program to calculate the absolute difference between the sums of the two diagonals of a square matrix

Algorithm :

Step 1 Start the program

Step 2 Read the size of the square matrix

Step 3 Read all matrix elements

Step 4 Initialize primary diagonal sum as zero

Step 5 Initialize secondary diagonal sum as zero

Step 6 Add elements where row index and column index are equal to primary diagonal sum

Step 7 Add elements where row index plus column index equals size minus one to secondary diagonal sum

Step 8 Find the absolute difference between both sums

Step 9 Display the difference

Step 10 Stop the program

Program :

```
import java.util.Scanner;
```

```
public class Main {  
    static int diagonalDifference(int[][] arr) {  
        int primary = 0;  
        int secondary = 0;  
        int n = arr.length;  
        for (int i = 0; i < n; i++) {
```

```
    primary += arr[i][i];  
    secondary += arr[i][n - 1 - i];  
}  
return Math.abs(primary - secondary);  
}
```

```
public static void main(String[] args) {  
    Scanner sc = new Scanner(System.in);  
    int n = sc.nextInt();  
    int[][] arr = new int[n][n];  
    for (int i = 0; i < n; i++) {  
        for (int j = 0; j < n; j++) {  
            arr[i][j] = sc.nextInt();  
        }  
    }  
    System.out.println(diagonalDifference(arr));  
    sc.close();  
}  
}
```

Sample Input :

3

11 2 4

4 5 6

10 8 -12

Sample Output :

15

Transpose Matrix

Aim :

To write a Java program to find the transpose of a matrix

Algorithm :

Step 1 Start the program

Step 2 Read the number of rows

Step 3 Read the number of columns

Step 4 Read all matrix elements

Step 5 Create a result matrix with columns as rows and rows as columns

Step 6 Visit each element of the original matrix

Step 7 Store each element at the switched row and column position in the result matrix

Step 8 Display the transpose matrix

Step 9 Stop the program

Program :

```
import java.util.Scanner;
```

```
public class Main {  
    static int[][] transpose(int[][] matrix) {  
        int rows = matrix.length;  
        int cols = matrix[0].length;  
        int[][] result = new int[cols][rows];  
        for (int i = 0; i < rows; i++) {  
            for (int j = 0; j < cols; j++) {  
                result[j][i] = matrix[i][j];  
            }  
        }  
    }  
}
```

```

    }
    return result;
}

public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);

    int rows = sc.nextInt();

    int cols = sc.nextInt();

    int[][] matrix = new int[rows][cols];

    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < cols; j++) {
            matrix[i][j] = sc.nextInt();
        }
    }

    int[][] result = transpose(matrix);

    for (int i = 0; i < result.length; i++) {
        for (int j = 0; j < result[0].length; j++) {
            System.out.print(result[i][j]);

            if (j < result[0].length - 1) {
                System.out.print(" ");
            }
        }

        System.out.println();
    }

    sc.close();
}
}

```

Sample Input :

3 3

1 2 3

4 5 6

7 8 9

Sample Output :

1 4 7

2 5 8

3 6 9

Sample Input :

2 3

1 2 3

4 5 6

Sample Output :

1 4

2 5

3 6

Transpose Matrix

Aim :

To write a Java program to find the transpose of a matrix

Algorithm :

Step 1 Start the program

Step 2 Read the number of rows

Step 3 Read the number of columns

Step 4 Read all matrix elements

Step 5 Create a result matrix with columns as rows and rows as columns

Step 6 Visit each element of the original matrix

Step 7 Store each element at the switched row and column position in the result matrix

Step 8 Display the transpose matrix

Step 9 Stop the program

Program :

```
import java.util.Scanner;
```

```
public class Main {  
    static int[][] transpose(int[][] matrix) {  
        int rows = matrix.length;  
        int cols = matrix[0].length;  
        int[][] result = new int[cols][rows];  
        for (int i = 0; i < rows; i++) {  
            for (int j = 0; j < cols; j++) {  
                result[j][i] = matrix[i][j];  
            }  
        }  
    }  
}
```



```

    }

    return result;
}

public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);

    int rows = sc.nextInt();

    int cols = sc.nextInt();

    int[][] matrix = new int[rows][cols];

    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < cols; j++) {
            matrix[i][j] = sc.nextInt();
        }
    }

    int[][] result = transpose(matrix);

    for (int i = 0; i < result.length; i++) {
        for (int j = 0; j < result[0].length; j++) {
            System.out.print(result[i][j]);

            if (j < result[0].length - 1) {
                System.out.print(" ");
            }
        }

        System.out.println();
    }

    sc.close();
}
}

```

Sample Input :

3 3

1 2 3

4 5 6

7 8 9

Sample Output :

1 4 7

2 5 8

3 6 9

Sample Input :

2 3

1 2 3

4 5 6

Sample Output :

1 4

2 5

3 6

Aim :

To rotate an $M \times N$ 2D matrix in the anti-clockwise direction by R rotations and print the resulting matrix.

Algorithm :**1. Layer Identification:**

- The total number of concentric rectangular rings (layers) is $\min(M, N) / 2$.
- Let L be the layer index running from 0 to $(\min(M, N) / 2) - 1$.

2. Extract Elements:

- For each layer L , extract all elements in an anti-clockwise order into a 1D list:
 - Top boundary: (L, c) for $c = L$ to $N - 1 - L$
 - Right boundary: $(r, N - 1 - L)$ for $r = L + 1$ to $M - 1 - L$
 - Bottom boundary: $(M - 1 - L, c)$ for $c = N - 2 - L$ down to L
 - Left boundary: (r, L) for $r = M - 2 - L$ down to $L + 1$

3. Effective Rotation:

- Compute the perimeter size of the layer: $P = 2 \times (M - 2L) + 2 \times (N - 2L) - 4$.
- Calculate effective shifts: $k = R \pmod{P}$.

4. Place Rotated Elements Back:

- Shift the 1D list by k positions (the element at index $(i + k) \pmod{P}$ moves to index i , which corresponds to counter-clockwise movement along the boundary).
- Map the shifted values back into their original matrix layer coordinates in the same traversal order.

5. Output:

- Print the modified matrix row by row.

Program :

```
import java.util.ArrayList;
```

```
import java.util.List;

import java.util.Scanner;

public class MatrixRotation {

    public static void matrixRotation(int[][] mat, int m, int n, int r) {

        int layers = Math.min(m, n) / 2;

        for (int layer = 0; layer < layers; layer++) {

            List<Integer> ring = new ArrayList<>();

            // 1. Extract Top row (left to right)
            for (int col = layer; col < n - layer; col++) {

                ring.add(mat[layer][col]);

            }

            // 2. Extract Right column (top to bottom)
            for (int row = layer + 1; row < m - layer; row++) {

                ring.add(mat[row][n - 1 - layer]);

            }

            // 3. Extract Bottom row (right to left)
            for (int col = n - 2 - layer; col >= layer; col--) {

                ring.add(mat[m - 1 - layer][col]);

            }

            // 4. Extract Left column (bottom to top)
            for (int row = m - 2 - layer; row > layer; row--) {

                ring.add(mat[row][layer]);

            }

        }

    }

}
```

```

// Effective rotation

int p = ring.size();

int shift = r % p;

// Put back rotated elements

int idx = shift;

// 1. Fill Top row
for (int col = layer; col < n - layer; col++) {
    mat[layer][col] = ring.get(idx % p);
    idx++;
}

// 2. Fill Right column
for (int row = layer + 1; row < m - layer; row++) {
    mat[row][n - 1 - layer] = ring.get(idx % p);
    idx++;
}

// 3. Fill Bottom row
for (int col = n - 2 - layer; col >= layer; col--) {
    mat[m - 1 - layer][col] = ring.get(idx % p);
    idx++;
}

// 4. Fill Left column
for (int row = m - 2 - layer; row > layer; row--) {
    mat[row][layer] = ring.get(idx % p);
    idx++;
}
}

```

```
}
```

```
public static void main(String[] args) {
```

```
    Scanner sc = new Scanner(System.in);
```

```
    if (!sc.hasNextInt()) return;
```

```
    int m = sc.nextInt();
```

```
    int n = sc.nextInt();
```

```
    int r = sc.nextInt();
```

```
    int[][] mat = new int[m][n];
```

```
    for (int i = 0; i < m; i++) {
```

```
        for (int j = 0; j < n; j++) {
```

```
            mat[i][j] = sc.nextInt();
```

```
        }
```

```
    }
```

```
    matrixRotation(mat, m, n, r);
```

```
    // Print output
```

```
    for (int i = 0; i < m; i++) {
```

```
        for (int j = 0; j < n; j++) {
```

```
            System.out.print(mat[i][j] + (j == n - 1 ? "" : " "));
```

```
        }
```

```
        System.out.println();
```

```
    }
```

```
    sc.close();
```

```
}
```

}

Sample Inputs & Outputs :

Sample 0 :

Plaintext

Input:

4 4 1

1 2 3 4

5 6 7 8

9 10 11 12

13 14 15 16

Output:

2 3 4 8

1 7 11 12

5 6 10 16

9 13 14 15

Sample 1 ;

Plaintext

Input:

4 4 2

1 2 3 4

5 6 7 8

9 10 11 12

13 14 15 16

Output:

3 4 8 12

2 11 10 16

1 7 6 15

5 9 13 14

Sample 2 :

Plaintext

Input:

5 4 7

1 2 3 4

7 8 9 10

13 14 15 16

19 20 21 22

25 26 27 28

Output:

28 27 26 25

22 9 15 19

16 8 21 13

10 14 20 7

4 3 2 1

Sample 3 :

Plaintext

Input:

2 2 3

1 1

1 1

Output:

1 1

1 1